

Práctica: Comparativa de Roles y Proveedores (Groq & Gemini)

Objetivo:

Construir un asistente que actúe como un **"Revisor de Código Senior"** utilizando Groq para velocidad y Gemini para un análisis profundo, observando cómo cambia la respuesta según el `system prompt`.

SISTEMA DE MENSAJES (ROLES)

Para interactuar con LLMs de forma profesional, no enviamos simplemente un texto, sino una **lista de mensajes estructurados**. Cada mensaje tiene un **rol**, lo que permite al modelo entender el contexto y su propósito.

ROL	ANALOGÍA	DESCRIPCIÓN Y USO
<code>system</code>	El Director	Define la personalidad , reglas y límites del asistente. Se configura al inicio y dicta cómo debe comportarse el modelo.
<code>user</code>	El Cliente	Son las instrucciones o preguntas que tú envías. Es el disparador de la acción.
<code>assistant</code>	La Memoria	Son las respuestas que el modelo generó previamente. Sirve para que la IA "recuerde" lo que se dijo antes.

Ejemplo de Estructura Multi-turno

Para que un chat tenga "memoria", enviamos un arreglo como este:

```
messages = [  
    # 1. Definimos las reglas  
    {  
        "role": "system",  
        "content": "Eres un asistente sarcástico pero eficiente que programa en Python."  
    }
```

```
    },
    # 2. El usuario pregunta
    {
        "role": "user",
        "content": "Hola, me llamo Alex. ¿Cómo hago un bucle infinito?"
    },
    # 3. La IA responde (esta respuesta se guarda para el siguiente turno)
    {
        "role": "assistant",
        "content": "Hola Alex, si quieres ver tu CPU arder, usa: while True: pass. ¿Algo más?"
    },
    # 4. El usuario vuelve a preguntar
    {
        "role": "user",
        "content": "¿Cómo dijiste que me llamaba?"
    }
]
```

Regla: El rol `system` siempre debe ir en la primera posición del array para establecer la base del razonamiento del modelo.

1. Preparación del Entorno

Crea una carpeta para el proyecto y dentro un archivo `.env`:

```
# .env
GROQ_API_KEY=tu_clave_groq
GEMINI_API_KEY=tu_clave_gemini
```

Instala las librerías necesarias:

```
pip install groq google-genai python-dotenv
```

2. Ejercicio 1: El impacto del Rol system (Groq)

En este ejercicio compararemos una petición "plana" con un rol de sistema definido. Usaremos el modelo llama-3.3-70b-versatile.

Crea un archivo llamado practica_groq.py:

```
import os
from groq import Groq
from dotenv import load_dotenv

load_dotenv()
client = Groq()

codigo_a_revisar = "def suma(a, b): return a+b"

# CONFIGURACIÓN A: Sin rol de sistema (comportamiento genérico)
res_generica = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[
        {"role": "user", "content": f"Revisa este código:
{codigo_a_revisar}"}
    ]
)

# CONFIGURACIÓN B: Con rol de sistema (comportamiento experto)
res_experta = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[
        {
            "role": "system",
            "content": "Eres un Ingeniero de Software Senior en
Google. Tu tono es crítico, técnico y breve. Solo respondes con
sugerencias de optimización o errores."
        },
        {"role": "user", "content": f"Revisa este código:
{codigo_a_revisar}"}
    ]
)

print("--- RESPUESTA GENÉRICA ---")
print(res_generica.choices[0].message.content)
print("\n--- RESPUESTA EXPERTA (SYSTEM ROLE) ---")
```

```
print(res_experta.choices[0].message.content)
```

Reto 1: Observa cómo en la respuesta B el modelo deja de saludarte y va directo al grano (debido al `system`).

3. Ejercicio 2: Memoria y el Rol `assistant` (Groq)

Para que la IA "recuerde", debemos enviarle el historial usando el rol `assistant`.

```
# Continuando en el mismo archivo o uno nuevo
historial = [
    {"role": "system", "content": "Eres un tutor de Python."},
    {"role": "user", "content": "Hola, me llamo Carlos y quiero aprender listas."},
    {"role": "assistant", "content": "¡Hola Carlos! Encantado. Una lista es una colección ordenada. ¿Quieres ver un ejemplo?"},
    {"role": "user", "content": "¿Cómo me llamo y qué estábamos hablando?"}
]

response = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=historial
)

print("\n--- PRUEBA DE MEMORIA ---")
print(response.choices[0].message.content)
```

4. Ejercicio 3: Implementación con Gemini 2.5

Ahora usaremos la nueva SDK de Google (`google-genai`) para realizar una tarea de razonamiento complejo con `gemini-2.5-flash`.

Crea el archivo `practica_gemini.py`:

```
from google import genai
from dotenv import load_dotenv
import os

load_dotenv()
```

```
# El cliente detecta GEMINI_API_KEY automáticamente
client = genai.Client()

# Ejemplo de uso con Gemini 2.5 Flash
prompt_tecnico = """
Explica el concepto de 'Decoradores' en Python.
Incluye un ejemplo de código y cómo se aplicaría en un entorno real
(ej. logging).
Responde en formato Markdown.
"""

response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=prompt_tecnico
)

print("--- ANÁLISIS DE GEMINI 2.5 ---")
print(response.text)
```

5. Tarea Final: El Comparador Multimodal

Crea un script final que haga lo siguiente:

1. Pregunte al usuario un concepto de programación.
2. Use **Groq** (con un rol de `system` sarcástico) para dar una definición rápida.
3. Use **Gemini** (modelo `gemini-2.5-pro`) para dar una explicación académica y detallada.